Skill progress monitoring

Catching robot failures while they happen

Pinhas Nehoray Aburmad

Ben-Gurion University of the Negev
Supervisor: Prof. Ronen Brafman



Skill progress monitoring
Catching robot failures while they happen

Pinhas Nehoray Aburmad

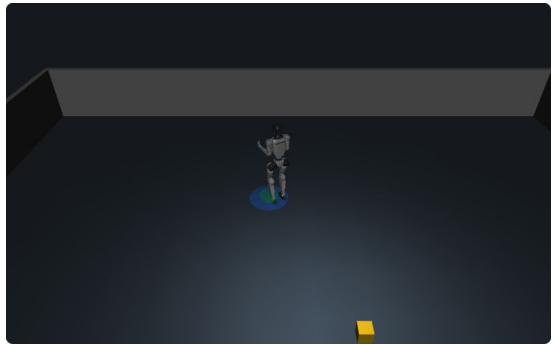
Ben-Gurion University of the Negev
Supervisor: Prof. Ronen Brafman



- Topic: watching a robot while it does a task, and noticing failures *during* the task.
- The talk: the problem, related work, our idea and its math, how we built and tested it, results, what is next.
- About 10 minutes. The live demo is ready if there is time (one command, see slide “What we learned”).

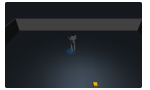
The problem

- Robots often fail **in the middle** of a task.
- They often **do not notice**.
- They report “done” anyway.
- **Goal:** notice the failure **the moment it happens**.



The robot says “done” at the drop-off (blue). The box (yellow) is still on the floor.

The problem
- Robots often fail **in the middle** of a task.
- They often **do not notice**.
- They report “done” anyway.
- **Goal:** notice the failure **the moment it happens**.



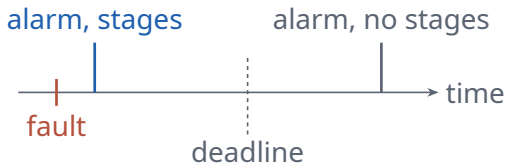
The robot says “done” at the drop-off (blue). The box (yellow) is still on the floor.

- The picture is a real run from our simulator: the box slipped while carrying; the robot walked on and reported success.
- The robot’s own program only knows what it *commanded*, not what really happened.
- Finding out at the end is late: the time is spent, and some failures cannot be undone.
- So we want an independent watcher that sees the failure when it happens.

Research question

RQ: Does giving the monitor the **stages** of a skill make it catch failures earlier, and does that improve success?

- **H1:** stage failures are caught **earlier**.
- **H2:** with a deadline, **more tasks succeed**; without one, no change.
- **H3:** it catches failures that **hand-written checks** miss.
- **H0:** for failures every stage shares (falls), **no change**.



Latency ℓ = time from the fault to the alarm.
Re-planning must finish before the deadline.

Research question
RQ: Does giving the monitor the stages of a skill make it catch failures earlier, and does that improve success?
H1: stage failures are caught earlier.
H2: with a deadline, more tasks succeed; without one, no change.
H3: it catches failures that hand-written checks miss.
H0: for failures every stage shares (falls), no change.
alarm, stages
alarm, no stages
time
fault
deadline
Latency ℓ = time from the fault to the alarm. Re-planning must finish before the deadline.

- One question, three hypotheses, and one null expectation stated before running.
- H1 is about *when*: latency ℓ from fault to alarm. H2 is about *outcome*: early only matters if there is a deadline (value of information). H3 is about *coverage* beyond what engineers foresaw.
- H0 is important for credibility: we say in advance where stages should *not* help (falls break “upright”, which every stage checks).
- The picture: with stages the alarm comes right after the fault; without, only at the end — after the deadline.

Related work: two streams

1. Skill description

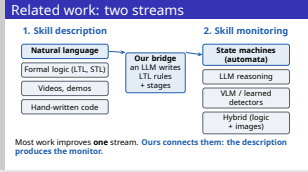
- **Natural language**
- Formal logic (LTL, STL)
- Videos, demos
- Hand-written code

Our bridge
an LLM writes
LTL rules
+ stages

2. Skill monitoring

- **State machines (automata)**
- LLM reasoning
- VLM / learned detectors
- Hybrid (logic + images)

Most work improves **one** stream. **Ours connects them: the description produces the monitor.**



- The field moves in two streams. (1) **How do we describe a skill?** In words, in logic, by showing videos or demonstrations, or in code. (2) **How do we watch the skill run** using that description? With state machines (formal), with an LLM that reasons in text, with a vision model trained on failures, or a mix.
- Most papers improve one stream: better translation from words to logic, or a better failure detector.
- Our path (blue): describe the skill in plain words; an LLM writes LTL rules *and* the stages; the monitor runs them as state machines on the robot's own sensors. So the description *is* the monitor.
- Next two slides: each stream, with examples and what each way is missing. Details: prep sheet §2–3.

Stream 1: describing a skill

Way	Examples	What is missing
Natural language	[Grounding Complex Natural Language Commands for Temporal Tasks in Unseen Environments, Liu et al., 2023] [nl2spec: Interactively Translating Unstructured Natural Language to Temporal Logics with LLMs, Cosler et al., 2023] [NL2TL: Transforming Natural Languages to Temporal Logics using LLMs, Chen et al., 2023] [NL2LTL: A Python Package for Converting Natural Language Instructions to LTL Formulas, Fuggitti, Chakraborti, 2023] [GinSign: Grounding Natural Language Into System Signatures for Temporal Logic Translation, English et al., 2025]	Made for planning or writing specs; no stages; words must be tied to sensors
Formal logic (LTL, STL)	[Runtime Verification for LTL and TLTL, Bauer, Leucker, Schallhart, 2011] [RTAMT: Runtime Robustness Monitors with Application to CPS and Robotics, Yamaguchi, Hoxha, Ničković, 2024]	An expert writes every rule; flat rules, no stages
Videos / demos	[Learning Task Specifications from Demonstrations, Vazquez-Chanlatte et al., 2018] [RoboCLIP: One Demonstration is Enough to Learn Robot Policies, Sontakke et al., 2023]	Need examples; the result is hard to read or edit
Hand-written code	Our baseline: checks written inside the robot program (Experiment 2)	Someone must foresee every failure

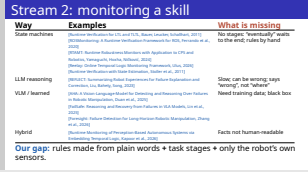
Stream 1: describing a skill

- **Natural language → logic.** Lang2LTL (CoRL 2023): route commands to LTL, GPT-4 finds the places, a small T5 writes the formula; works in new cities. nl2spec (CAV 2023): LLM translation with sub-translations a human can fix (16/36 alone, 31/36 with a human). NL2TL (EMNLP 2023): lifted translation with a fine-tuned T5, 28k pairs. NL2LTL (AAAI 2023 demo): picks one of a few fixed patterns; a demo paper, no benchmark of its own. GinSign (2025): grounds the placeholder words to a real system's signature; 100% predicate F1 on its three domains. All target planning or spec writing; none adds stages or watches a running robot.
- **Formal logic by experts.** LTL (Bauer et al.) and STL with robustness (RTAMT). Precise, but someone must write every rule, and the rules are flat.
- **Videos / demonstrations.** Vazquez-Chanlatte et al. (NeurIPS 2018) infer a logical specification from demonstrations (maximum-entropy likelihood, search over candidates). RoboCLIP (NeurIPS 2023) uses one video or text description and a pretrained video-language model as a reward; 2–3× better zero-shot than other imitation methods. They need examples, and the result is hard to read or edit.
- **Code.** Our hand-written checks baseline: strong on what was foreseen (81% on time), silent on what was not (40/40). Prep sheet §2.

Stream 2: monitoring a skill

Way	Examples	What is missing
State machines	[Runtime Verification for LTL and TLTL, Bauer, Leucker, Schallhart, 2011] [ROSMonitoring: A Runtime Verification Framework for ROS, Ferrando et al., 2020] [RTAMT: Runtime Robustness Monitors with Application to CPS and Robotics, Yamaguchi, Hoxha, Ničković, 2024] [Reelay: Online Temporal Logic Monitoring Framework, Ulus, 2026] [Runtime Verification with State Estimation, Stoller et al., 2011]	No stages: “eventually” waits to the end; rules by hand
LLM reasoning	[REFLECT: Summarizing Robot Experiences for Failure Explanation and Correction, Liu, Bahety, Song, 2023]	Slow; can be wrong; says “wrong”, not “where”
VLM / learned	[AHA: A Vision-Language-Model for Detecting and Reasoning Over Failures in Robotic Manipulation, Duan et al., 2025] [FailSafe: Reasoning and Recovery from Failures in VLA Models, Lin et al., 2025] [Foresight: Failure Detection for Long-Horizon Robotic Manipulation, Zhang et al., 2026]	Need training data; black box
Hybrid	[Runtime Monitoring of Perception-Based Autonomous Systems via Embedding Temporal Logic, Kapoor et al., 2026]	Facts not human-readable

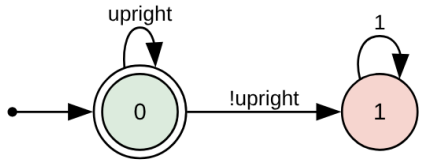
Our gap: rules made from plain words **+** task stages **+** only the robot’s own sensors.



- **State machines.** Bauer et al. (TOSEM 2011): LTL_3, true / false / “?”, smallest monitor, earliest verdict; we build on it. ROSMonitoring (TAROS 2020): ROS topics $\rightarrow$ monitor nodes, any logic as the oracle. RTAMT (STTT 2024): STL with robustness, ROS integration, Toyota HSR case study. Reelay (2026): fast past-time logic monitoring, about a million messages per log in benchmarks. Stoller et al. (RV 2011): an HMM fills gaps in observation and gives the probability a property holds. None knows the task’s stages; all need rules written by hand.
- **LLM reasoning.** REFLECT (CoRL 2023): summarises sensor data into a text history; an LLM explains why a task failed and suggests a fix (RoboFail dataset). Our MiniGrid pilot measured the risk: a free-text LLM judge gave $+0$ while the LTL monitor gave $+1.00$.
- **VLM / learned.** AHA (ICLR 2025): a VLM fine-tuned on generated failures; beats GPT-4o by 10.3%. FailSafe (2025): failures made in simulation, a VLM proposes fixes. Foresight (2026): world-model latents + conformal alarm. All need failure data and give no readable reason.
- **Hybrid.** Embedding TL (2026): logic whose facts are distances to reference images in an embedding space. Prep sheet §3.

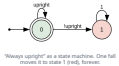
Background: what is a monitor?

- A program that **watches** the robot's messages.
- It checks **rules over time**, written in **LTL** (a logic of time).
- Example: "*always* stay upright".
- Each rule becomes a small **state machine**.
- Problem: "*eventually* deliver" **never fails early**.



"Always upright" as a state machine. One fall moves it to state 1 (red), forever.

Background: what is a monitor?
- A program that **watches** the robot's messages.
- It checks **rules over time**, written in **LTL** (a logic of time).
- Example: "*always* stay upright".
- Each rule becomes a small **state machine**.
- Problem: "*eventually* deliver" **never fails early**.

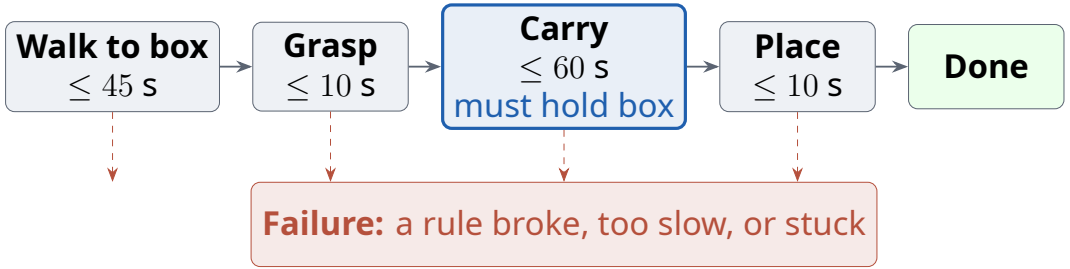


"Always upright" as a state machine. One fall moves it to state 1 (red), forever.

- LTL = Linear Temporal Logic: rules with "always" (G) and "eventually" (F).
- The tool Spot turns each rule into a small automaton. Every second the monitor feeds it the facts it measured (upright? holding the box?) and takes one step.
- "Always upright" fails the moment the robot falls: easy.
- "Eventually deliver" can never fail early, because delivery might still happen later. This is the weakness we fix next.

Our idea: the stages of the task

Each stage has its own rules and its own time limit.



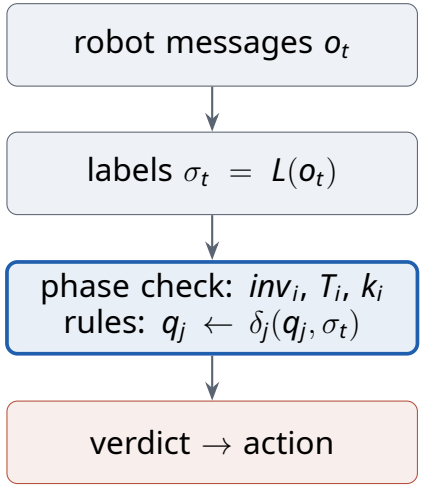
While carrying, the robot **must hold the box**. A drop is caught **at once**.

Our idea: the stages of the task
Each stage has its own rules and its own time limit.
Walk to box ≤ 45 s
Grasp ≤ 10 s
Carry ≤ 60 s must hold box
Place ≤ 10 s
Done
Failure: a rule broke, too slow, or stuck
While carrying, the robot must hold the box. A drop is caught at once.

- We call these stages “progress states”. Each stage says: what must stay true, what ends the stage, a time limit, and that the robot must keep making progress.
- These stages were written by the LLM from the plain description of “fetch and deliver”.
- Key example: in Carry, “holding the box” must stay true. A drop breaks it in the next second.
- The same fact (“not holding”) is normal while walking to the box. A rule without stages cannot say “must hold”, because it would be false at the start.

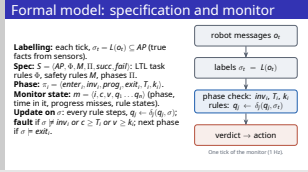
Formal model: specification and monitor

Labelling: each tick, $\sigma_t = L(o_t) \subseteq AP$ (true facts from sensors).
Spec: $S = \langle AP, \Phi, M, \Pi, succ, fail \rangle$: LTL task rules Φ, safety rules M, phases Π.
Phase: $\pi_i = \langle enter_i, inv_i, prog_i, exit_i, T_i, k_i \rangle$.
Monitor state: $m = \langle i, c, v, q_1 \dots q_n \rangle$ (phase, time in it, progress misses, rule states).
Update on σ: every rule steps, $q_j \leftarrow \delta_j(q_j, \sigma)$; **fault** if $\sigma \not\models inv_i$ *or* $c \geq T_i$ *or* $v \geq k_i$; next phase if $\sigma \models exit_i$.



One tick of the monitor (1 Hz).

Formal model: specification and monitor



- Labelling L: the evaluator turns the messages of one second into true/false facts. Example: holding $\Leftrightarrow$ box ≥ 0.3 m high and ≤ 0.5 m from the base.
- Spec: the LLM writes all of it — facts, LTL rules, safety rules, phases, end conditions.
- Phase: enter, invariant (must hold all the time), progress, exit, time limit T_i, progress-miss limit k_i (3 here).
- Update, in words: every rule takes one step; the current phase fails if its invariant is false, or its time is up, or progress failed k times; it moves on when its exit holds.
- This is exactly what the code does (monitor_node.py: “hard invariant — immediate failure”). Full write-up: formal_model.md.

Model predictions vs. measurements

Property (from the model)	Prediction	Measured
P1. A broken stage invariant fails *at the tick it is seen*.	Drop caught at once.	**1 tick** (20/20)
P2. "Eventually" rules have *no failing state*: any prefix can still succeed.	LTL alone misses the drop until the end.	LTL only: 14 ticks
P3. A stuck stage fails within its time limit T_i.	Failed grasp caught by Grasp's 10 s.	**4 ticks** vs. 19
P4. "Holding" is false before the grasp, so it cannot be a whole-task rule.	Flat spec sees the drop only at the end.	Flat: 14 ticks
P5. Rules every stage shares are in the flat spec too.	Falls: no difference.	1 tick, all versions

The results follow from the model; they are not a lucky outcome.

Model predictions vs. measurements

Property (from the model)	Prediction	Measured
P1. A broken stage invariant fails *at the tick it is seen*.	Drop caught at once.	**1 tick** (20/20)
P2. "Eventually" rules have *no failing state*: any prefix can still succeed.	LTL alone misses the drop until the end.	LTL only: 14 ticks
P3. A stuck stage fails within its time limit T_i.	Failed grasp caught by Grasp's 10 s.	**4 ticks** vs. 19
P4. "Holding" is false before the grasp, so it cannot be a whole-task rule.	Flat spec sees the drop only at the end.	Flat: 14 ticks
P5. Rules every stage shares are in the flat spec too.	Falls: no difference.	1 tick, all versions

The results follow from the model; they are not a lucky outcome.

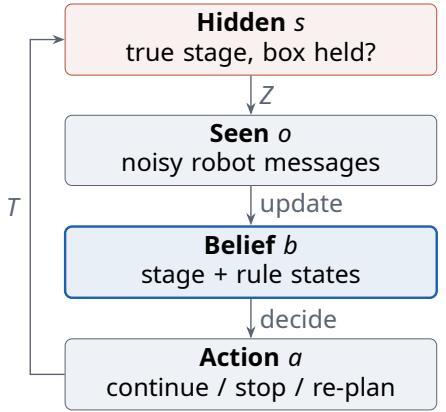
- P1 proof: the invariant is checked every tick and fails on the first false tick. Only the labelling delay remains.
- P2 proof: for $\varphi = F(a_1 \wedge F(a_2 \wedge \dots))$, any finite run that has not finished can still be extended to finish (re-grasp, deliver). So no finite run is a "bad prefix" and LTL_3 says "?" (Bauer et al. 2011). Checked: Spot's automaton has 6 states, one accepting, no rejecting state.
- P3 proof: the time-in-phase counter grows by one per tick and the phase fails at T_i.
- P4: the flat invariant is "upright and not too close" — no "holding".
- Honest: phases do not add expressiveness; "G(grasped $\rightarrow$ (holding U near_dropoff))" is LTL. The point is that stages make such rules explicit, so the LLM writes them.

The decision view: a POMDP

- **Hidden** s: what really happens (stage, box held?).
- **Seen** o: the robot's messages.
- **Belief** b: our best guess of s.
- **Action** a: continue, stop, or re-plan.

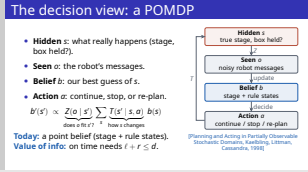
$$b'(s') \;\propto\; \underbrace{Z(o \mid s')}_{\text{does } o \text{ fit } s'?} \sum_{s} \underbrace{T(s' \mid s, a)}_{\text{how } s \text{ changes}} \; b(s)$$

Today: a point belief (stage + rule states).
Value of info: on time needs $\ell + r \le d$.



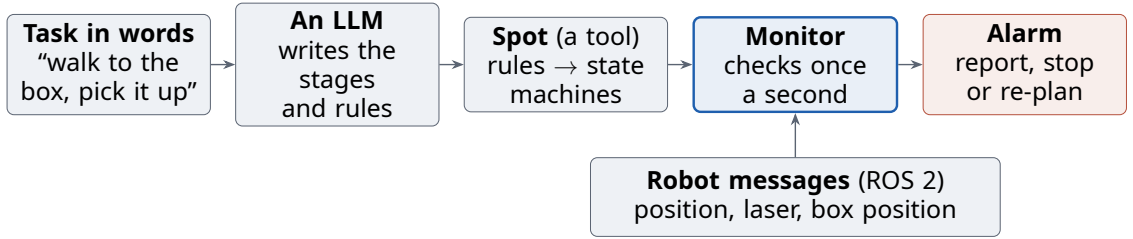
[Planning and Acting in Partially Observable Stochastic Domains, Kaelbling, Littman, Cassandra, 1998]

The decision view: a POMDP



- POMDP (Kaelbling et al., AI Journal 1998): a hidden state, noisy observations, a belief, and actions chosen from the belief.
- The formula, right to left: move the old guess forward with how the world changes (T), then weigh each state by how well it explains what we saw (Z).
- Why stages help: the stage is part of s. "Not holding" fits "walking" but not "carrying", so in Carry one observation shows the failure.
- Honest: today the guess is one point, the update one automaton step, the action a fixed rule. No probabilities yet (last step of "To be done").
- Value of information: early knowledge helps only if it changes the outcome; tested with deadlines in Experiment 2.

Method: how the monitor is built and run



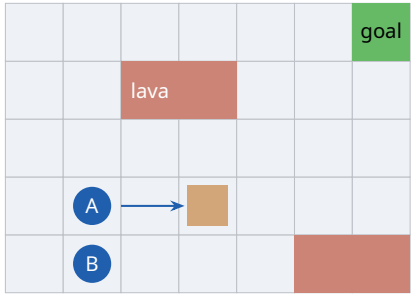
- The rules are written **automatically**, from the task in words.
- The monitor uses **only sensors**. It never asks the planner.

Method: how the monitor is built and run
Task in words “walk to the box, pick it up” → An LLM writes the stages and rules → Spot (a tool) rules → state machines → Monitor checks once a second → Alarm report, stop or re-plan
Robot messages (ROS 2) position, laser, box position
- The rules are written **automatically**, from the task in words.
- The monitor uses **only sensors**. It never asks the planner.

- The LLM (Claude) wrote the whole spec in one try; automatic checks made sure it loads and that every fact it uses can be measured. It was saved before any experiment ran.
- An “evaluator” turns the raw messages into true/false facts each second, e.g. “holding the box” is inferred from where the box is relative to the robot.
- Evaluator and monitor are separate ROS 2 programs, exactly as on a real robot.
- The monitor never reads the planner’s own status: a test enforces this. (Experiment 1 shows why.)

First test bed: continuous MiniGrid + MAAOS

- Box pushing in a **continuous MiniGrid**, with **MAAOS** (Fouzi's multi-agent planner; I integrated it).
- One agent, or two agents pushing together.
- Wrong plan or deadlock: monitor + re-plan **fixed every seed**.
- A free-text **LLM judge: no gain**. It says "wrong", not *where* in the task.
- No false alarms on clean runs.



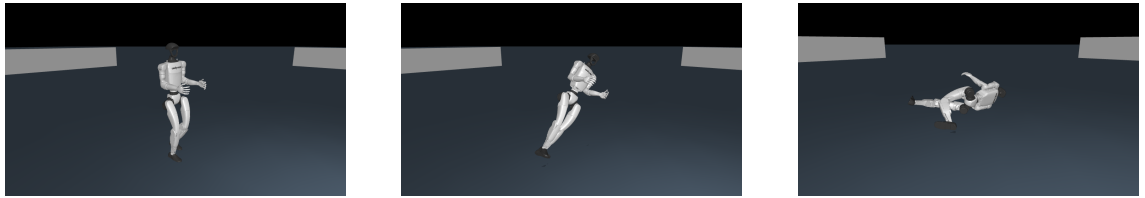
Two agents (A, B) push a heavy box to the goal and avoid lava.

First test bed: continuous MiniGrid + MAAOS
• Box pushing in a **continuous MiniGrid**, with **MAAOS** (Fouzi's multi-agent planner; I integrated it).
• One agent, or two agents pushing together.
• Wrong plan or deadlock: monitor + re-plan **fixed every seed**.
• A free-text **LLM judge: no gain**. It says "wrong", not *where* in the task.
• No false alarms on clean runs.
Two agents (A, B) push a heavy box to the goal and avoid lava.

- Our first test bed, now frozen as a pilot. MAAOS is Fouzi's multi-agent project; I integrated it into a continuous version of MiniGrid (agents move and push with continuous dynamics; lava tiles are hazards).
- Tasks: one agent pushes a heavy box to a goal; two agents must reach the box, line up on the same side and push together.
- Scenarios: clean plan, hazard navigation, an injected fault (corrupted plan / coordination deadlock). Arms: monitor off, detect only, detect + re-plan. 10 seeds per cell for the rule-based monitors, 5 for the LLM-backed ones.
- Results: corrupted plan 0 $\rightarrow$ every seed with re-plan ($\Delta = +1.00$); deadlock: off fails every seed (80-step budget), LTL + re-plan recovers every seed in about 45 steps. Detect only: $+0.00$ everywhere. Free-text LLM judge: $+0.00$ even with re-plan — first evidence for stages. Clean runs: 1.00 success, 0 false alarms.
- Honest: scripted reference plans, not the live MAAOS planner; discrete-mode episodes. One spec fault was mine (an over-strict alignment rule), fixed by relaxing the formula; a stronger judge did not help.

Test bed: a humanoid robot in MuJoCo

- Unitree **G1** robot in the **MuJoCo** physics simulator.
- We **cause failures on purpose**: push, stall, drop, failed grasp.
- We compare the monitor **with stages** and **without**.



Walking → pushed → fallen. The simulator knows the truth; the monitor does not.

Test bed: a humanoid robot in MuJoCo
- Unitree **G1** robot in the **MuJoCo** physics simulator.
- We **cause failures on purpose**: push, stall, drop, failed grasp.
- We compare the monitor **with stages** and **without**.

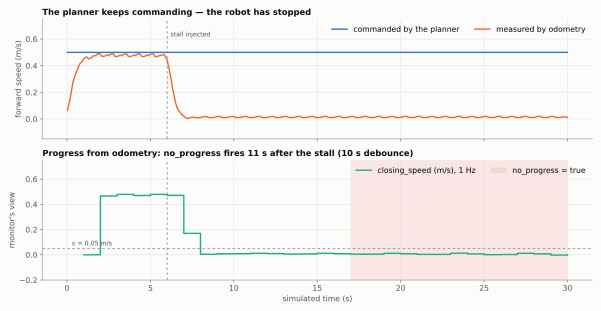


Walking → pushed → fallen. The simulator knows the truth; the monitor does not.

- The G1 walks with Unitree's own released walking controller.
- Each failure is injected at a known time, so we can measure how fast it is caught.
- "Without stages" = the same spec with all stages merged into one (a fair control: same facts, same safety rules).
- Success or failure is judged by the simulator's ground truth, which the monitor never sees.

Experiment 1: a short walk (1800 runs)

- Stages helped **little** here: 15 vs. 14 of 15 stalls caught.
- What mattered: measure progress **from the robot's sensors**.
- Without that: only **1 of 15** caught.



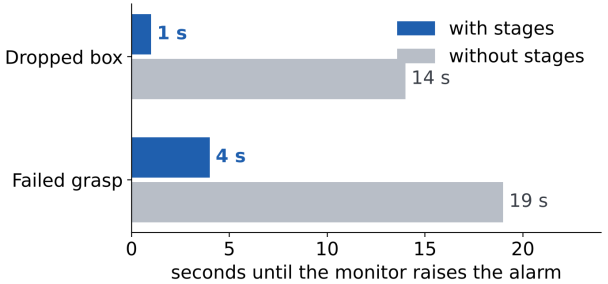
The planner still says "go". The robot has stopped. The monitor sees no progress.

- 10 failure types × 20 seeds × 3 versions × 3 reactions = 1800 runs.
- Walking has two stages that check the same things, so stages add little: falls are caught in 1–7 s by every version.
- Stalls are the difference. "No progress" is measured from odometry against the goal; without it only 1 of 15 stalls is caught.
- The figure: the planner keeps commanding 0.5 m/s; the robot has stopped. A monitor that trusted the planner would see nothing wrong.

Experiment 1: a short walk (1800 runs)
- Stages helped **little** here: 15 vs. 14 of 15 stalls caught.
- What mattered: measure progress **from the robot's sensors.**
- Without that: only **1 of 15** caught.

Experiment 2: fetch and deliver (3360 runs)

- Four stages.
- Box dropped: caught after **1 s** with stages, 14 s without.
- Grasp failed: **4 s** vs. 19 s.
- Earlier in **all 20** runs.



Middle value (median) of 20 runs for each failure.

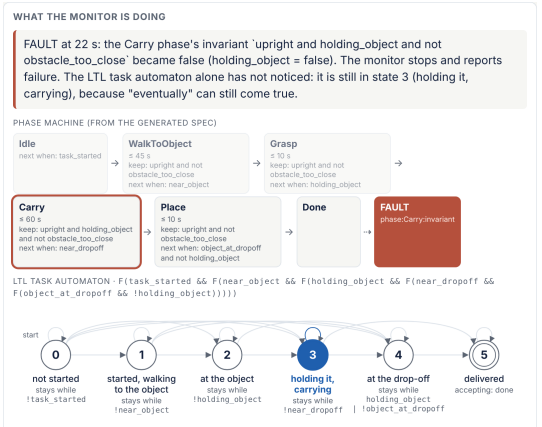
Experiment 2: fetch and deliver (3360 runs)
• Four stages.
• Box dropped: caught after 1 s with stages, 14 s without.
• Grasp failed: 4 s vs. 19 s.
• Earlier in all 20 runs.
Middle value (median) of 20 runs for each failure.

- Walk to a box, grasp it, carry it 2.5–4 m, place it. Evaluator and monitor ran as separate ROS 2 programs; the box position came from a noisy detector (2 cm noise, 5% missed).
- Drop: caught 1 s after it happens, because Carry requires holding. Without stages the monitor only notices when the robot claims success, 14 s later.
- Failed grasp: caught by the Grasp stage's time limit (4 s vs. 19 s).
- Paired by scenario: stages caught it earlier in all 20 runs, later in none.

One run, second by second

- **21.6 s:** the box drops.
- **22 s:** the Carry rule breaks. Alarm.
- The plain LTL rule still waits: “maybe later”.

The stage caught it.



From the live demo (one run). The current stage and state are lit.

One run, second by second
- **21.6 s:** the box drops.
- **22 s:** the Carry rule breaks. Alarm.
- The plain LTL rule still waits: “maybe later”.

The stage caught it.

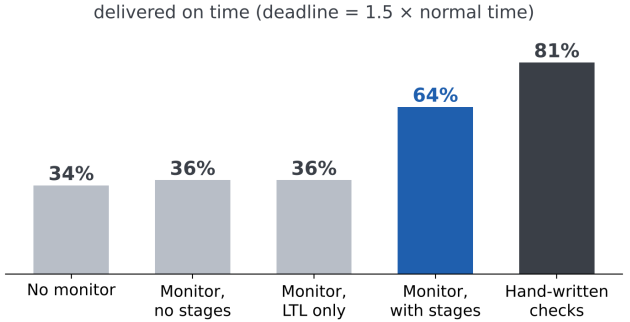


From the live demo (one run). The current stage and state are lit.

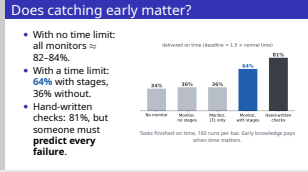
- This is one run from the live demo page (seed 0), not a statistic.
- Top: the stages, Carry outlined in red. Bottom: the 6-state LTL automaton for “walk, grasp, carry, place, in order”.
- The automaton is still in state 3 (“holding, carrying”): it has no “failed” state, because “eventually” can always still come true.
- This is the whole idea in one picture: the stage catches what the LTL rule alone cannot.

Does catching early matter?

- With no time limit: all monitors ≈ 82–84%.
- With a time limit: **64%** with stages, 36% without.
- Hand-written checks: 81%, but someone must **predict every failure**.



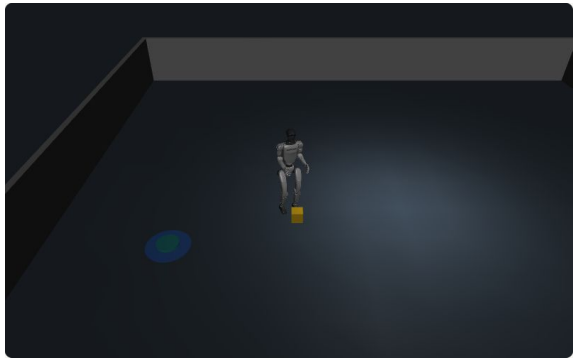
Tasks finished on time, 160 runs per bar. Early knowledge pays when time matters.



- On an alarm the robot re-plans. With no deadline, every monitor ends at 82–84% success, because a late re-plan still works.
- With a deadline of 1.5× the normal time: 64% with stages vs. 36% without. LTL alone (no stages): 36%.
- This is the “value of information” from the POMDP slide: early knowledge pays only when time matters.
- Hand-written checks inside the robot program do better (81%), because the program knows its own state. But an engineer had to foresee each failure: see the next slide.

Failures nobody planned for

- Box left at the wrong place; robot too close to a wall.
- Hand-written checks: **40 of 40** missed.
- With our monitor: **0 of 40** missed.
- Missed by all: too slow, or a long detour.



The box is far from the drop-off (blue), yet the robot says “done”. Our monitor catches it.

Failures nobody planned for
- Box left at the wrong place; robot too close to a wall.
- Hand-written checks: 40 of 40 missed.
- With our monitor: 0 of 40 missed.
- Missed by all: too slow, or a long detour.
The box is far from the drop-off (blue), yet the robot says “done”. Our monitor catches it.

- New failures, one per requirement of the task description that the hand-written checks do not cover. The design and predictions were written down before the run.
- Wrong drop-off: the robot’s own idea of the drop-off is wrong, so its own check passes and it reports success. Too close: the navigator grazes obstacles.
- Hand-written checks alone: 40 of 40 failed silently. Adding our monitor: 0 of 40, at no measurable cost elsewhere.
- Missed by everyone: running slowly, or a detour, that stay inside the time limits. Our prediction for “slow” was wrong, and we say so.

Hypotheses and results

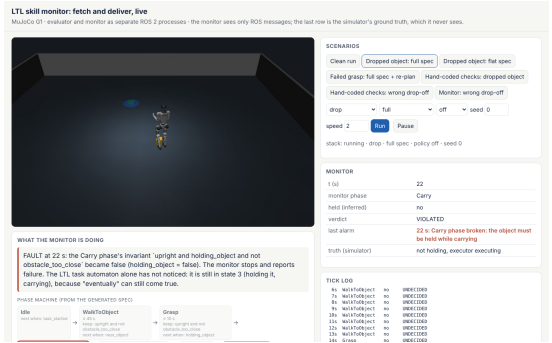
Hypothesis	Result	Verdict
H1: earlier detection	drop 1 vs. 14 s; grasp 4 vs. 19 s; 20/20 runs	**supported**
H2: more on-time success	64% vs. 36% with a deadline; 82% vs. 84% without	**supported**
H3: beyond hand-written checks	40/40 missed $\rightarrow$ 0/40	**supported**
H0: falls unchanged	1 s in every version	as predicted
Not caught	too slow, detours, skipped pick-up	reported

Hypotheses and results
Hypothesis | Result | Verdict
H1: earlier detection | drop 1 vs. 14 s; grasp 4 vs. 19 s; 20/20 runs | supported
H2: more on-time success | 64% vs. 36% with a deadline; 82% vs. 84% without | supported
H3: beyond hand-written checks | 40/40 missed → 0/40 | supported
H0: falls unchanged | 1 s in every version | as predicted
Not caught | too slow, detours, skipped pick-up | reported

- Each hypothesis was written down before the run; the table is the answer to each.
- How sure: earlier in every one of the 20 paired runs; 64% vs. 36% over 160 runs per version. Report counts, not tests.
- The misses are reported, with their reason: progress means “closer to where the robot walks” and the time limits are generous. Fix: progress toward each stage’s own target, calibrated limits.

What we learned

- **Stages catch failures when they happen.**
- Early alarms help **when time matters**.
- Rules come from words, so failures **nobody planned for** are caught too.
- Use the **robot's own sensors**, not the planner.

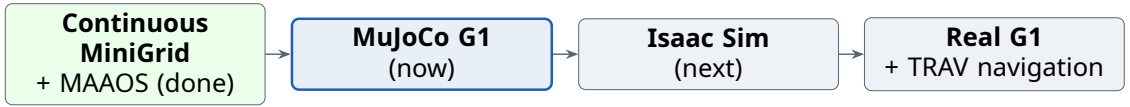


Live demo, one command:
`docker run -p 8080:8080 ghcr.io/pinhas019/ltlmonitor-demo`

What we learned
- Stages catch failures when they happen.
- Early alarms help when time matters.
- Rules come from words, so failures nobody planned for are caught too.
- Use the robot's own sensors, not the planner.
Live demo, one command:
docker run -p 8080:8080
ghcr.io/pinhas019/ltlmonitor-demo

- Four take-aways, one per line on the slide.
- If there is time, show the demo: open http://localhost:8080, click "Dropped object: full spec", press Pause at 22 s.
- The demo page draws the stages and the automaton live, from the monitor's own ROS 2 messages.

To be done



- **Fix what we found:** better time limits; progress toward each stage's goal.
- **More tests:** more tasks, more rule sets, more failures.
- **The full POMDP:** camera, real probabilities, best decisions.

Limits today: grasp and camera are simulated; thresholds not tuned.

- Platforms: continuous MiniGrid + MAAOS (pilot, done) → MuJoCo G1 (now, real dynamics) → Isaac Sim (richer scenes, cameras, more skills) → the real G1 with Elias's TRAV navigation app (monitor on vs. off).
- What stays the same at every step: one monitor engine, the same sensor vocabulary; only the robot's "adapter" changes. The real G1, MuJoCo and Isaac adapters already expose the same sensor schema.
- Fix: time limits from normal runs; progress toward each stage's own target (catches a skipped pick-up); restart at the current stage after re-plan.
- Paper 2, the full POMDP: camera facts with probabilities, a real belief, stop / continue / re-plan by expected cost.
- Limits: the legs-only G1 cannot grasp (box attached); box position from a simulated detector.

References I

[Runtime Verification for LTL and TLTL, Bauer, Leucker, Schallhart, 2011]. ACM TOSEM 20.

[NL2TL: Transforming Natural Languages to Temporal Logics using Large Language Models, Chen, Gandhi, Zhang, Fan, 2023]. EMNLP.

[nl2spec: Interactively Translating Unstructured Natural Language to Temporal Logics with Large Language Models, Cosler, Hahn, Mendoza, et al., 2023]. CAV.

[AHA: A Vision-Language-Model for Detecting and Reasoning Over Failures in Robotic Manipulation, Duan et al., 2025]. ICLR.

[GinSign: Grounding Natural Language Into System Signatures for Temporal Logic Translation, English, Walker, Simon, Ewetz, 2025]. arXiv.

[ROSMonitoring: A Runtime Verification Framework for ROS, Ferrando, Cardoso, Fisher, Ancona, et al., 2020]. TAROS.

[NL2LTL: A Python Package for Converting Natural Language Instructions to LTL Formulas, Fuggitti, Chakraborti, 2023]. AAAI (demo).

[Planning and Acting in Partially Observable Stochastic Domains, Kaelbling, Littman, Cassandra, 1998]. Artificial Intelligence 101.

[Runtime Monitoring of Perception-Based Autonomous Systems via Embedding Temporal Logic, Kapoor, Hammer, Kapoor, et al., 2026]. arXiv.

[FailSafe: Reasoning and Recovery from Failures in Vision-Language-Action Models, Lin, Duan, Fang, Fox, et al., 2025]. arXiv.

[Grounding Complex Natural Language Commands for Temporal Tasks in Unseen Environments, Liu, Yang, Idrees, Liang, et al., 2023]. CoRL.

[REFLECT: Summarizing Robot Experiences for Failure Explanation and Correction, Liu, Bahety, Song, 2023]. CoRL.

[RoboCLIP: One Demonstration is Enough to Learn Robot Policies, Sontakke, Zhang, Arnold, et al., 2023]. NeurIPS.

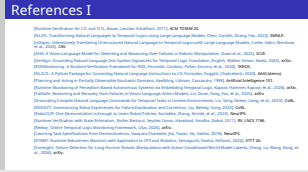
[Runtime Verification with State Estimation, Stoller, Bartocci, Seyster, Grosu, Havelund, Smolka, Zadok, 2011]. RV, LNCS 7186.

[Reelay: Online Temporal Logic Monitoring Framework, Ulus, 2026]. arXiv.

[Learning Task Specifications from Demonstrations, Vazquez-Chanlatte, Jha, Tiwari, Ho, Seshia, 2018]. NeurIPS.

[RTAMT: Runtime Robustness Monitors with Application to CPS and Robotics, Yamaguchi, Hoxha, Ničković, 2024]. STTT 26.

[Foresight: Failure Detection for Long-Horizon Robotic Manipulation with Action-Conditioned World Model Latents, Zhang, Lu, Wang, Kang, et al., 2026]. arXiv.



Every entry is a link. Sources: the verified BibTeX in `papers/`; Kaelbling 1998, Stoller 2011, Vazquez-Chanlatte 2018, RoboCLIP, REFLECT and AHA confirmed by web search. FailSafe, Foresight, Embedding-TL, GinSign and Reelay link to arXiv.